- lecture Link for Dropout: https://www.scaler.com/meetings/i/nn-regularisation-of-nns/archive (Timestamp = 1:13:13 - 1:53:13)
- Lecture Link BatchNorm: https://www.scaler.com/meetings/i/nn-tensorflow-and-keras-1/archive (Timestamp = 1:09:53 - 1:51:16)
- Lecture Link Regularization: https://www.scaler.com/meetings/i/nn-tensorflow-and-keras-1/archive (Timestamp = 0:05:11 - 0:13:11)

Outline

- Business Case
- EDA
- Baseline Model
- Regularization
  - Fraudulent transaction classification
  - Model capacity
- Dropout
  - Dropout in neural networks
  - Parameters after dropout
  - Output for the query point
  - Weights trained in an epoch
  - Masked vector
- Batch Normalization
  - Batch normalization
  - Batch normalization characteristics
- EarlyStopping

Business Case

Amazon is facing a high surge of returns on some of its Products, which has led to the downgrade of the company credibility.

So they have appointed you as their Data Scientist:

- To estimate whether customer will return the product or not
- based on the product description, transportation, importance and prices

With this, lets load the data:

```
import numpy as np

import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

def create_baseline():

  model = Sequential([
                   Dense(256, activation="relu"),
                   Dense(128, activation="relu"),
                   Dense(64, activation="relu"),
                   Dense(1 , activation = 'sigmoid')])
  return model

model = create_baseline()

def custom_mse(y_true, y_pred):
  loss = np.square(y_pred - y_true)
  loss = loss * [0.5, 0.5]            #x
  loss = np.sum(loss, axis=0)          #y
  return loss

model.compile(loss=custom_mse, optimizer='adam')


import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns



!gdown 1mMRZKe5Qm99fJBE9y0mLdvVZYDfHvkxY

df = pd.read_csv('Amazon.csv', encoding='latin-1')
df.dropna(inplace = True)
```

```
Downloading...
From: https://drive.google.com/uc?id=1mMRZKe5Qm99fJBE9y0mLdvVZYDfHvkxY
To: /content/Amazon.csv
100% 429k/429k [00:00<00:00, 108MB/s]
```

Data Description:

Id	Features	Description
01	**ID**	ID of Customers
02	**Warehouse_block**	The Company have big Warehouses which is divided in to block such as A,B,C,D,E
03	**Mode_of_Shipment**	The Company Ships the products in multiple way such as Ship, Flight and Road.
04	**Customer_care_calls**	The number of calls made from enquiry for enquiry of the shipment
05	**Customer_rating**	The company has rated from every customer. 1 is the lowest (Worst), 5 is the highest (Best)
06	**Cost_of_the_Product**	Price of the Product

Id	Features	Description
07	**Prior_purchases**	The Number of Prior Purchases of the customer
08	**Product_importance**	The company has categorized the product in the various parameter such as low, medium, high.
09	**Gender**	If Customer is a Male or Female
10	**Discount_offered**	Discount offered on that specific product
11	**Weight_in_gms**	It is the weight in grams
12	**Returned**	It is the target variable, where 1 Indicates that the product is returned

```
df.head()
```

	ID	Warehouse_block	Mode_of_Shipment	Customer_care_calls	Customer_rating
0	1	D	Flight	4	2
1	2	F	Flight	4	5
2	3	A	Flight	2	2
3	4	B	Flight	3	3
4	5	C	Flight	2	2

Total Number of samples and features of the data:

Records	Features
10999	12

```
df.shape
```

```
(10999, 12)
```

```
X = df.drop(columns=['ID','Returned'])
y = df['Returned']
```

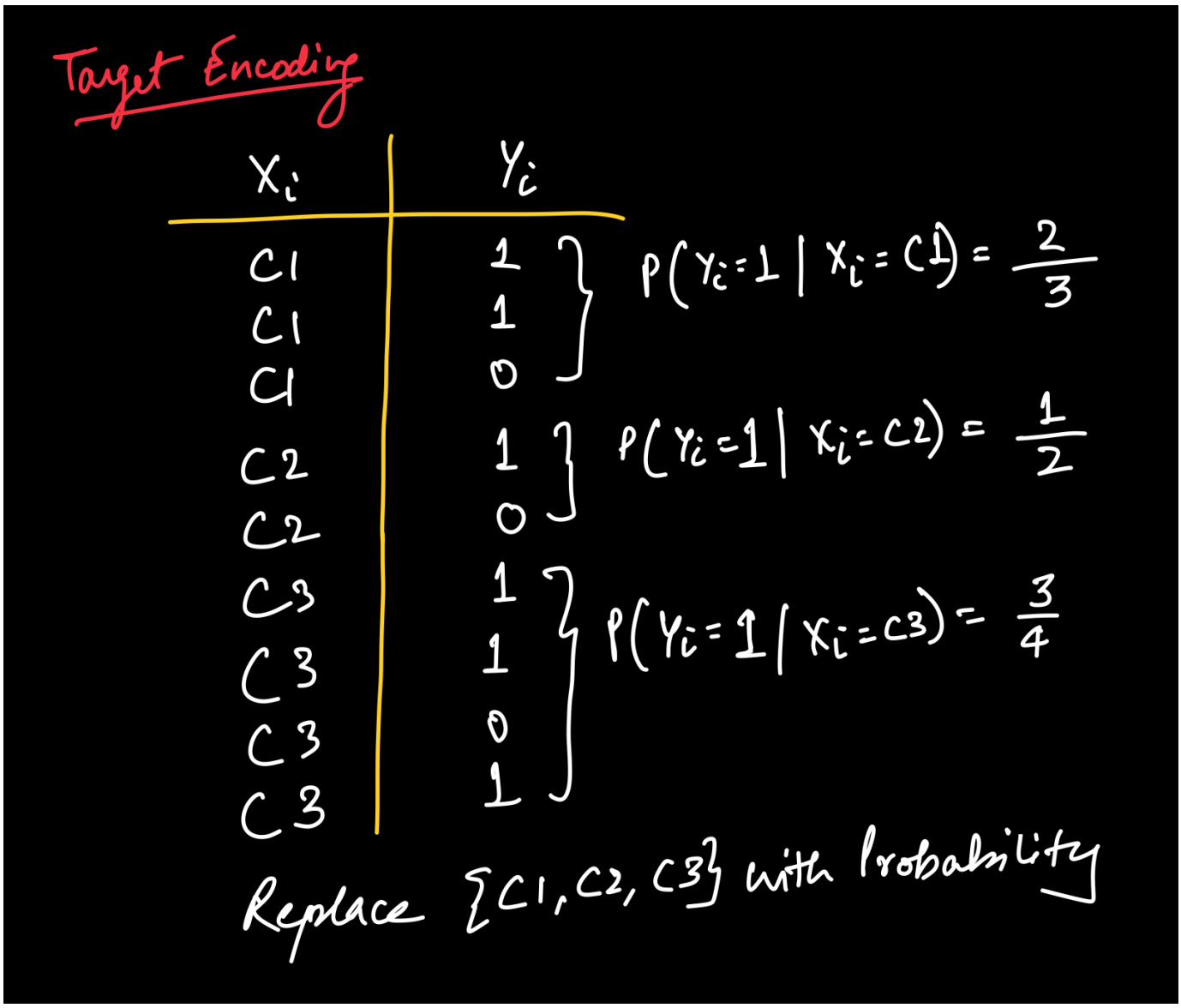
Splitting the data into Train, Validation and Test Data

```
from sklearn.model_selection import train_test_split

X_train_val, X_test, y_train_val, y_test = train_test_split(X, y, test_size=0.2,
X_train, X_val, y_train, y_val = train_test_split(X_train_val, y_train_val, test_

print('Train : ', X_train.shape, y_train.shape)
print('Validation:', X_val.shape, y_val.shape)
print('Test  : ', X_test.shape, y_test.shape)
```

```
Train :  (7039, 10) (7039,)
Validation: (1760, 10) (1760,)
Test  :  (2200, 10) (2200,)
```

EDA

Instructor Note: Covered in PreRead



Which Encoding to use to transform Categorical features ?

Ans: Since there are many categories in all the Categorical features,

- Which will make the One-Hot Encoding quite large and sparse
- Its better to use Target Encoding

How do target encoding ?

Ans: Target Encoding finds the Probability of $y =$(some class$k$) when the Categorical feature $(f) = C_i$

- $P(y = k|f = C_i)$
- And replace the all the samples which have feature value as C_i to $P(y = k|f = C_i)$

```
Suppose y = [0,1] and f = [c1,c2,c3] such that:
- out of 200 samples, 50 samples have y = 1, when f = c1
-  25 samples have y = 1, when f = c2
-  And, 10 samples have y = 1, when f = c3
```

What is the Target Encoding value for the feature ?

Ans: $P(Y = 1|f = C_1) = \frac{50}{200} = 0.25$

- $P(Y = 1|f = C_2) = \frac{25}{200} = 0.13$
- $P(Y = 1|f = C_3) = \frac{10}{200} = 0.05$

With this, lets implement Target Encoding

```
!pip install category-encoders
```

```
Collecting category-encoders
  Downloading category_encoders-2.6.3-py2.py3-none-any.whl (81 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 81.9/81.9 kB 1.4 MB/s eta 0:00:(
Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.10/dis
Requirement already satisfied: scikit-learn>=0.20.0 in /usr/local/lib/python3
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.10/dist
Requirement already satisfied: statsmodels>=0.9.0 in /usr/local/lib/python3.1(
Requirement already satisfied: pandas>=1.0.5 in /usr/local/lib/python3.10/dis
Requirement already satisfied: patsy>=0.5.1 in /usr/local/lib/python3.10/dist
Requirement already satisfied: python-dateutil>=2.8.1 in /usr/local/lib/pytho
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.10/dis
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.10/d
Installing collected packages: category-encoders
Successfully installed category-encoders-2.6.3
```

```
from category_encoders import TargetEncoder
```

Converting all the Categorical Features to Numerical

- using TargetEncoding

```
enc = TargetEncoder(cols=['Warehouse_block','Mode_of_Shipment','Product_importanc
X_train = enc.fit_transform(X_train, y_train)

X_val = enc.transform(X_val, y_val)
X_test = enc.transform(X_test, y_test)

X_train.head()
```

	Warehouse_block	Mode_of_Shipment	Customer_care_calls	Customer_rating
10286	0.578005	0.608167	6	2
7746	0.600336	0.600251	4	3
1789	0.601109	0.608167	5	2
2521	0.601109	0.600251	6	4
10404	0.600336	0.576471	5	3

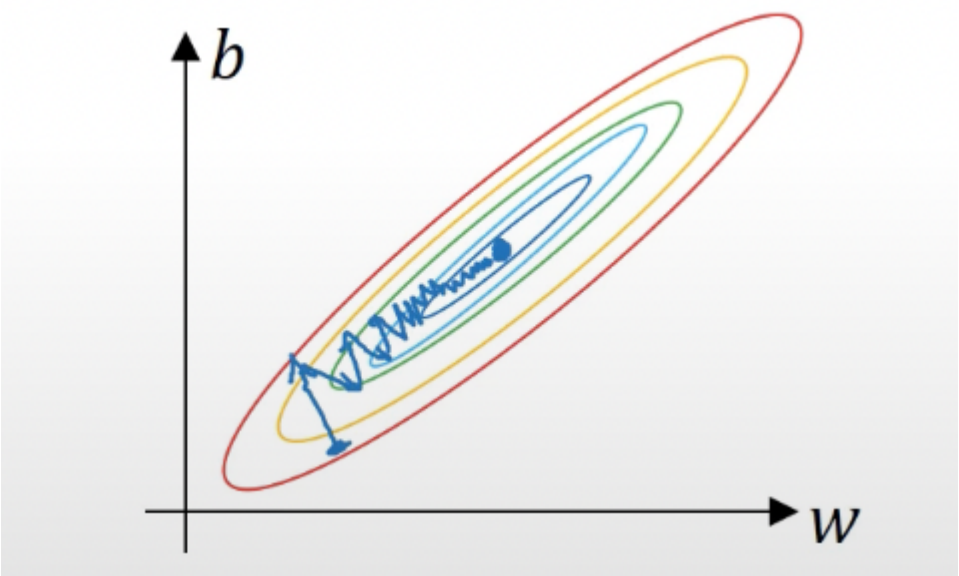
Standardization

Observe

- How the data is not Standardized

Lets Standardize the data

```
from sklearn.preprocessing import StandardScaler


scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)

X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)
```



What happens if the data is not standardized ?

Ans: if data is not standardized, then suppose:

- if one feature has a range from 1 to 1000, and the other has a range from 0 to 1
- then the weights w_1 and w_2 assosicated to these features will vary in value alot
    - With w_1 being a very high value as compared to w_2

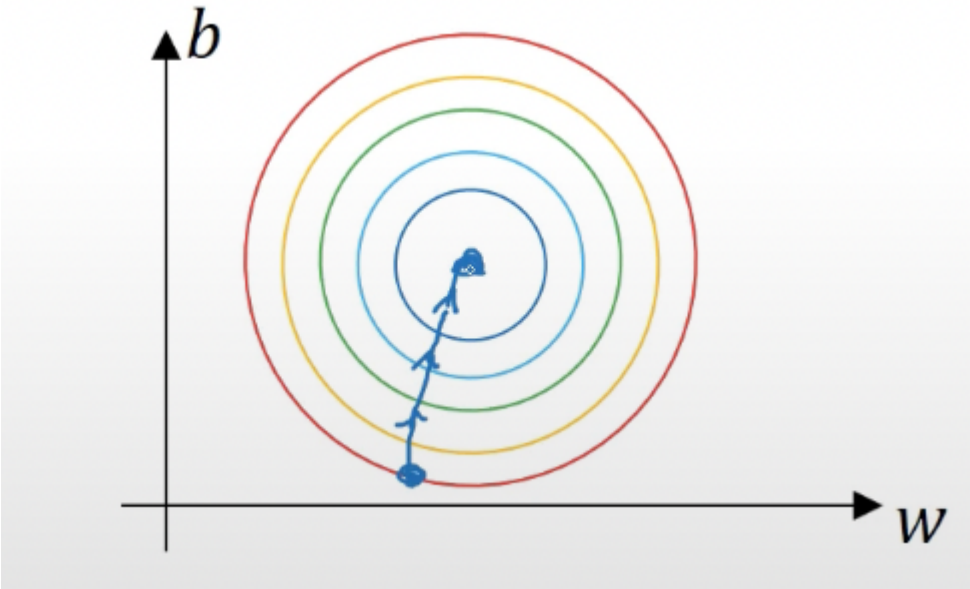
Thus the plot for Loss function ($L(w,b) = \frac{1}{2n}\sum_{i=1}^{i=n} L(\hat{y}_i, y_i)$) against the weights for n samples:

- Becomes a squashed, very unsymmetric and closed

Why there is an issue with a squashed and closed Loss function plot ?

Ans: A very small Learning Rate will be required for gradient Descent to reach the global minima

- thus increasing the number of steps and time



What happens when data is standardized ?

Ans: Since now the data is standardized, it makes:

- Range of both the feature 1 and feature 2 from 0 to 1

Thus making the weights for both the features close to each other

- Therefore the plot for Loss function ($L(w,b)$)becomes
    - Spread out and Symmetric

Why a Spread out and Symmetric Loss function plot works fine ?

Ans: We can use a much large Learning rate to quickly reach the global minima

Baseline Model

Now that the data is ready, we can implement

- A simple NN model using tensorflow keras
- By creating 5 layered NN such that:

Layer	Descrption
L1	Is the Input Layer
L2	Contains 256 Neurons
L3	Contains 128 Neurons
L4	Contains 64 Neurons
L5	Is the Output Layer

Note: In Between the layers, we will use:

- ReLU as the Activation function

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense


# For Reproducibility
np.random.seed(42)
tf.random.set_seed(42)


def create_baseline():

  model = Sequential([
                  Dense(256, activation="relu"),
                  Dense(128, activation="relu"),
                  Dense(64, activation="relu"),
                  Dense(1 , activation = 'sigmoid')])
  return model
```

What Loss to use when the final layer Neuron output $(\hat{y} = a_1^5) \in [0, 1]$ while $y \in \{0, 1\}$?

Ans: Recall from Logisitic Regression:

- when Sigmoid $\sigma(z) \in (0, 1)$ and $y = \in \{0, 1\}$
- LogLoss was used. $Logloss(L) = -y \times log(\hat{y}) - (1 - y) \times log(1 - \hat{y})$

Hence here too, LogLoss can be used.

Note: Logloss in Tensorflow Keras is implemented using `BinaryCrossentropy()`

```
model = create_baseline()
```

Using `Adam` as Optimizer and metric being `Accuracy`

```
model.compile(optimizer = tf.keras.optimizers.Adam(),
                loss = tf.keras.losses.BinaryCrossentropy(),
                metrics=["accuracy"])
```

Loading Tensorboard

```
%load_ext tensorboard

from datetime import datetime
now = datetime.now()
log_folder = "tf_logs/.../" + now.strftime("%Y%m%d-%H%M%S") + "/"
```

Clearing any running logs

```
!rm -rf logs
```

```
from tensorflow.keras.callbacks import TensorBoard

tb_callback = TensorBoard(log_dir=log_folder, histogram_freq=1)
```

Training the model with epoch=10 and batch size = 128

```
history = model.fit(X_train, y_train, validation_data = (X_val, y_val),  epochs=1
```

```
Epoch 1/10
55/55 [==============================] - 7s 30ms/step - loss: 0.5485 - accura
Epoch 2/10
55/55 [==============================] - 1s 11ms/step - loss: 0.5157 - accura
Epoch 3/10
55/55 [==============================] - 1s 11ms/step - loss: 0.5131 - accura
Epoch 4/10
55/55 [==============================] - 1s 16ms/step - loss: 0.5074 - accura
Epoch 5/10
55/55 [==============================] - 1s 12ms/step - loss: 0.5053 - accura
Epoch 6/10
55/55 [==============================] - 1s 10ms/step - loss: 0.5015 - accura
```

```
Epoch 7/10
55/55 [==============================] - 0s 8ms/step - loss: 0.4993 - accuracy
Epoch 8/10
55/55 [==============================] - 0s 8ms/step - loss: 0.4967 - accuracy
Epoch 9/10
55/55 [==============================] - 0s 8ms/step - loss: 0.4945 - accuracy
Epoch 10/10
55/55 [==============================] - 0s 8ms/step - loss: 0.4925 - accuracy
```

Lets check the Model performance on Training and Validation data

```
model.evaluate(X_train, y_train)
```

```
220/220 [==============================] - 1s 2ms/step - loss: 0.4826 - accura
[0.4826110899448395, 0.7106122970581055]
```

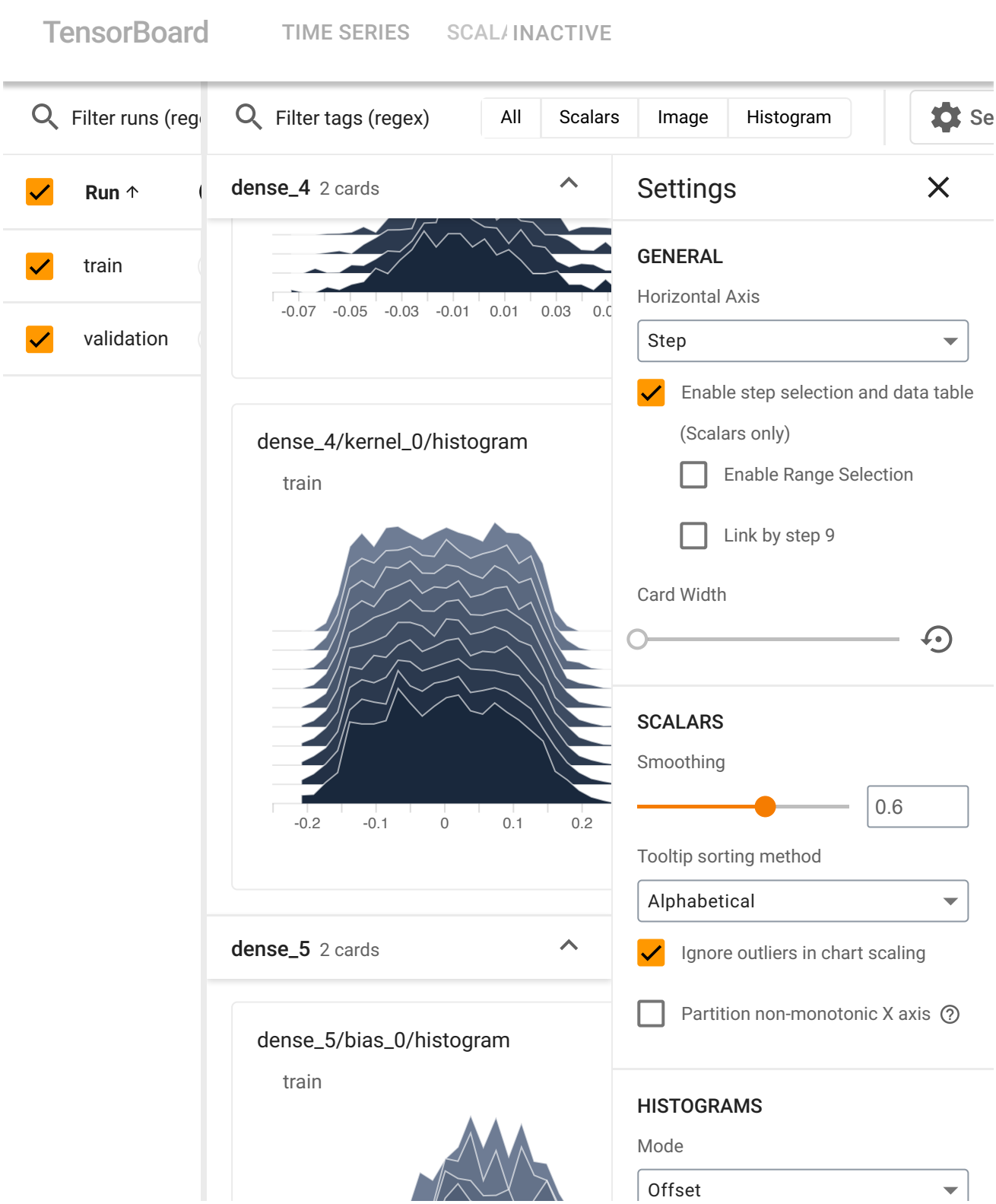
```
model.evaluate(X_val, y_val)
```

```
55/55 [==============================] - 0s 3ms/step - loss: 0.5454 - accuracy
[0.545381486415863, 0.6403409242630005]
```

Lets plot the Accuracy vs Epochs plots for both training and Validation

- for better clarity

```
%tensorboard --logdir={log_folder}
```



Observe

The model has:

- training accuracy : 69.9%
- and Validation accuracy : 64%

What can we understand from model's training accracy being 69.9% and Validaiton Accuracy being 64% ?

Ans: Clearly, the model overfits the data meaning:

- Model has a low Bias
- But a High Variance

How can we make the baseline model not Overfit ?

Ans: By using Regularization

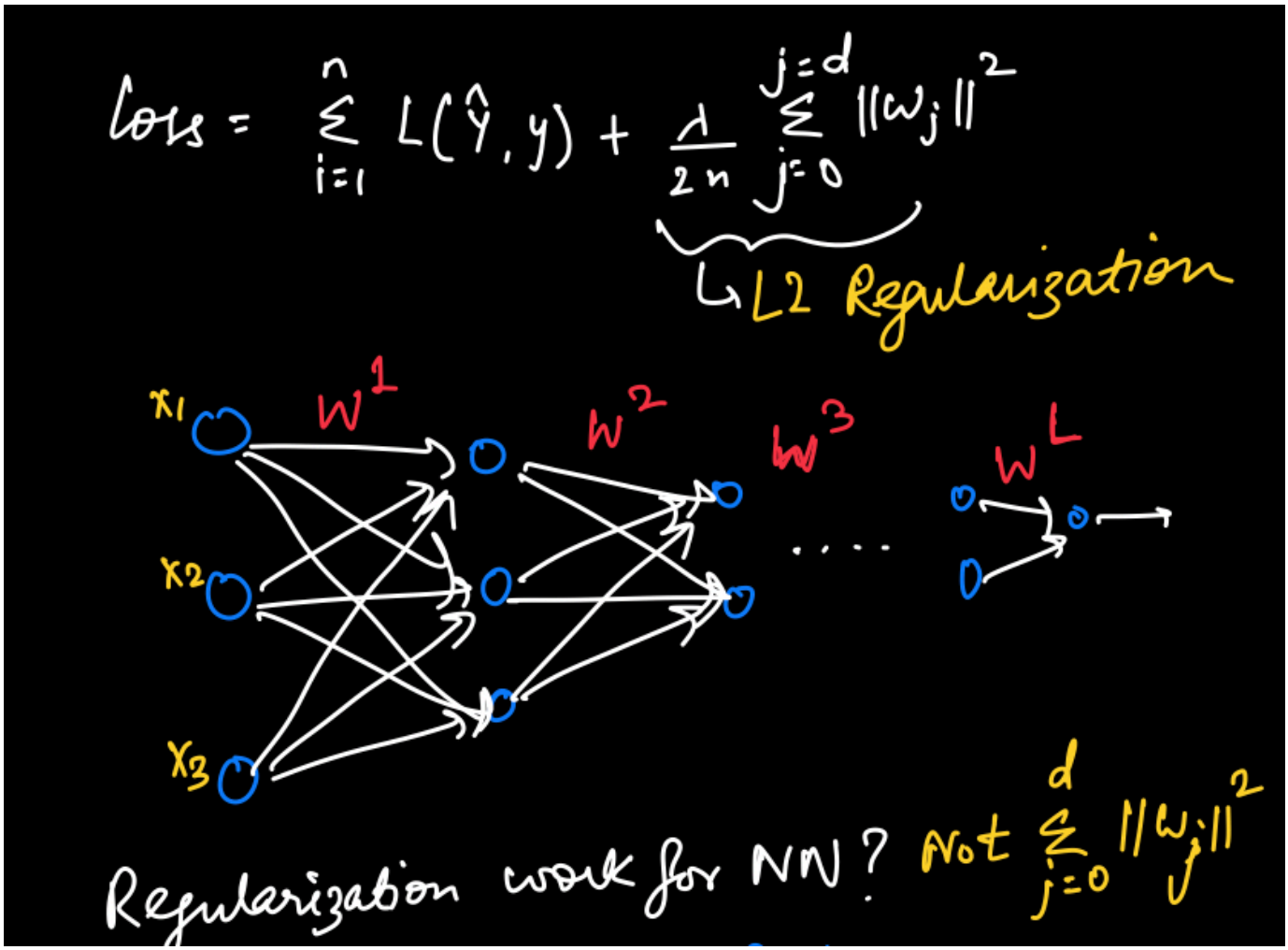
Quiz

```
Choose the correct option for when model underfits,

1. Model has high Bias and low Variance
2. Model has low Bias and low Variance
3. Model has high Bias and high Variance
```

Answer

1. Model has high Bias and low Variance

Regularization



What Regularization method to use ?

Ans: Recall from ML module, we used L-1 and L-2 regularization methods to

- resolve overfitting of the model

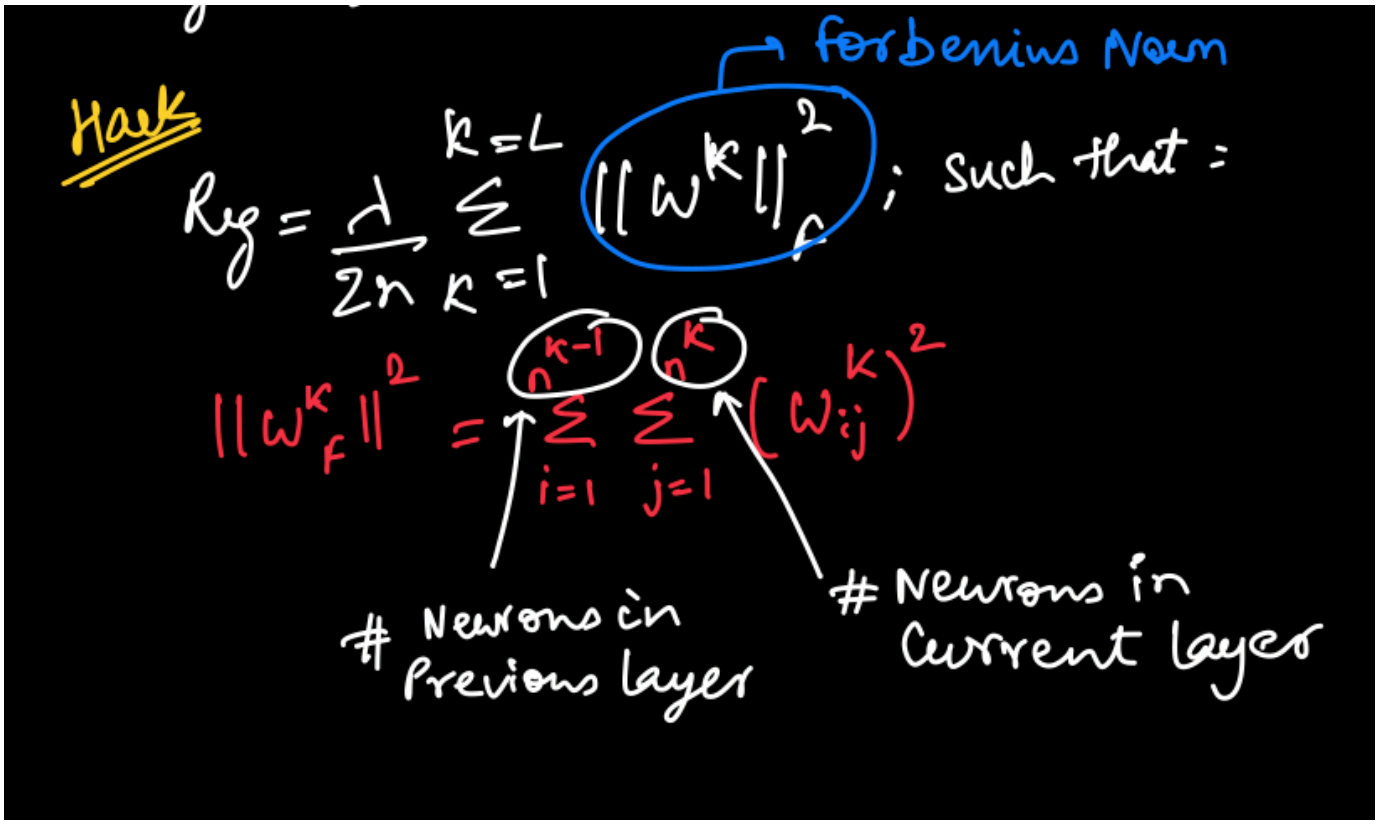
Also, L2 Regularization for n samples with d features is defined as:

- $L2 = \frac{\lambda}{2n} \sum_{j=0}^{i=d} ||w_j||_2^2$

Can we use L2 Regularization in NN ?

Ans: No, since in NN there is

- L Weight matrices for each layer $\{W^1, W^2, \ldots, W^L\}$
- While in Machine Learning there is only a single weight matrix W



How to do Regularization in NN ?

Ans: Now if we assume that

- the number of neurons in the current layer k is n^k
- and the number of neurons in the previous layer $k-1$ is n^{k-1}

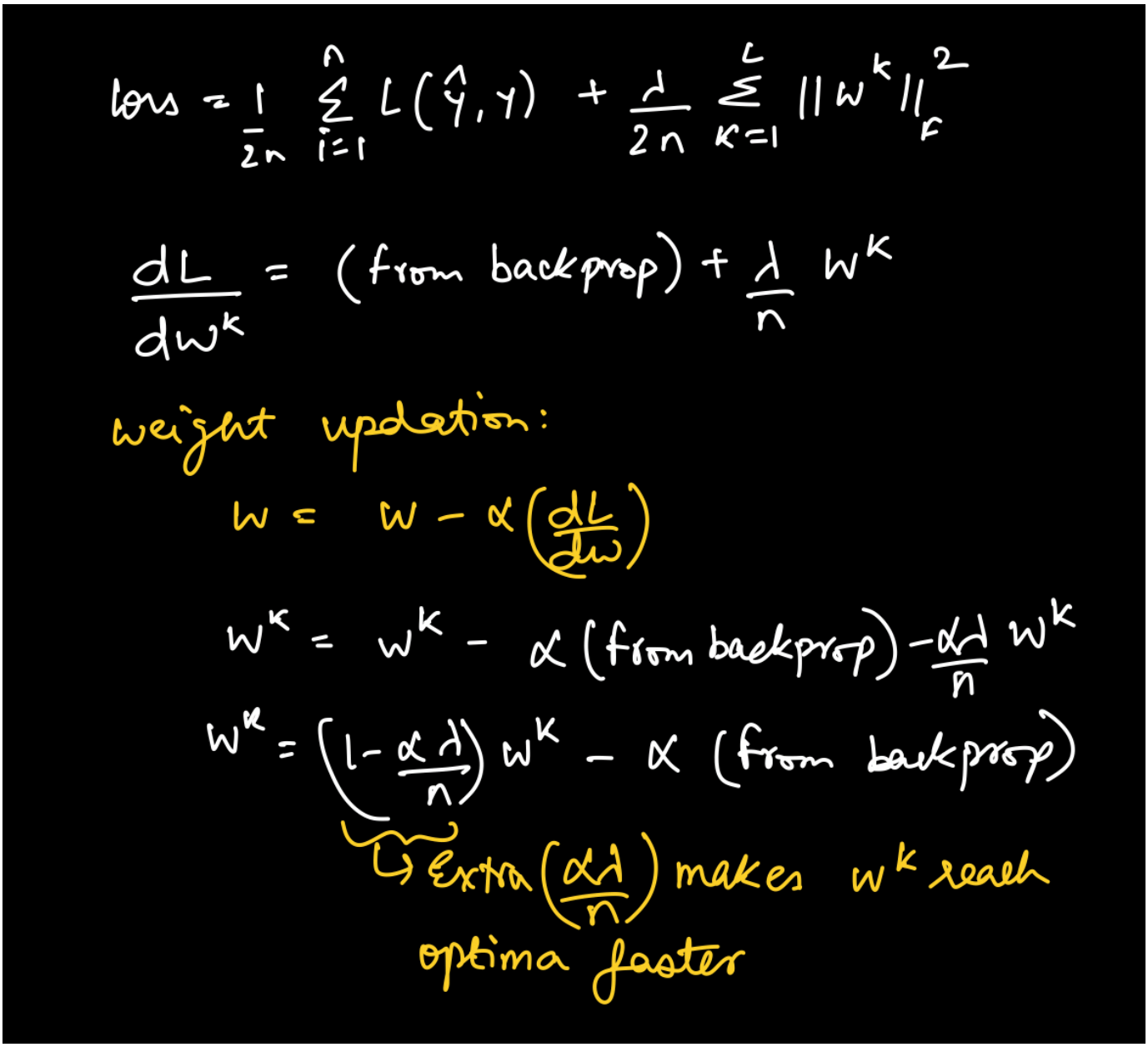
Then Regularization:

- $Reg = \frac{\lambda}{2n} \sum_{k=1}^{k=L} ||W^k||_F^2$

Where we define $||W^k||_F^2$ as:

- $\sum_{i=1}^{n^{k-1}} \sum_{j=1}^{n^k} (w_{ij}^k)^2$

Note: The term $||W||_F^2$ is called **Forbenius Norm**



How does weight updation occur with Regularization ?

Ans: With regularization the loss function will be defined as:

- $Loss = \frac{1}{2n} \sum_{i=1}^{i=n} L(\hat{y}_i, y_i) + \frac{\lambda}{2n} \sum_{k=1}^{k=L} ||W^k||_F^2$

What will be the gradient (dw^L) ?

Ans: dw^L = (From Backpropagation) $+\frac{\lambda}{n} W^L$

How do we do the weight updation ?

Ans: if Learning Rate is α, then:

- Weight updation: $W^L = W^L - \alpha \times dw^L$

Therefore:

- $W^L = W^L - \frac{\alpha\lambda}{n} W^L - \alpha$(From Backpropagation)

On Simpliying:

- $W^L = (1 - \frac{\alpha\lambda}{n}) \times W^L - \alpha$(From Backpropagation)

Note: Due to the extra $(1 - \frac{\alpha\lambda}{n})$,

- L2 Regularization is also called **Weight Decay**

Quiz

```
if a Network has 2 input neuron, 2 hidden neuron and 1 output neuron such that:
 W^1 = [4, 2, 3, 1] , then what is the value of ||W^1||^2_F

1. 10
2. 30
3. 20
```

Answer

2. 30

Explanation

$||W^1||_F^2 = \sum_{i=1}^{2} \sum_{j=1}^{2} (w_{ij}^1)^2$

- $||W^1||_F^2 = 4^2 + 2^2 + 3^2 + 1^2$
- $||W^1||_F^2 = 16 + 4 + 9 + 1 = 30$

Lets now implement L2 Regularization on the Baseline Model

```
def create_baseline():
    # lambda = 0.01
    L2Reg = tf.keras.regularizers.L2(l2=1e-6)
    model = Sequential([
                    Dense(256, activation="relu", kernel_regularizer = L2Reg ),
                    Dense(128, activation="relu", kernel_regularizer = L2Reg),
                    Dense(64, activation="relu", kernel_regularizer = L2Reg),
                    Dense(1 , activation = 'sigmoid')])
    return model


model = create_baseline()

model.compile(optimizer = tf.keras.optimizers.Adam(),
                loss = tf.keras.losses.BinaryCrossentropy(),
                metrics=["accuracy"])
```

Loading Tensorboard

```
!rm -rf logs
```

```
from tensorflow.keras.callbacks import TensorBoard

now = datetime.now()
log_folder = "tf_logs/.../" + now.strftime("%Y%m%d-%H%M%S") + "/"

tb_callback = TensorBoard(log_dir=log_folder, histogram_freq=1)
```

Training the model

```
history = model.fit(X_train, y_train, validation_data = (X_val, y_val),  epochs=1
```

Evaluating the model

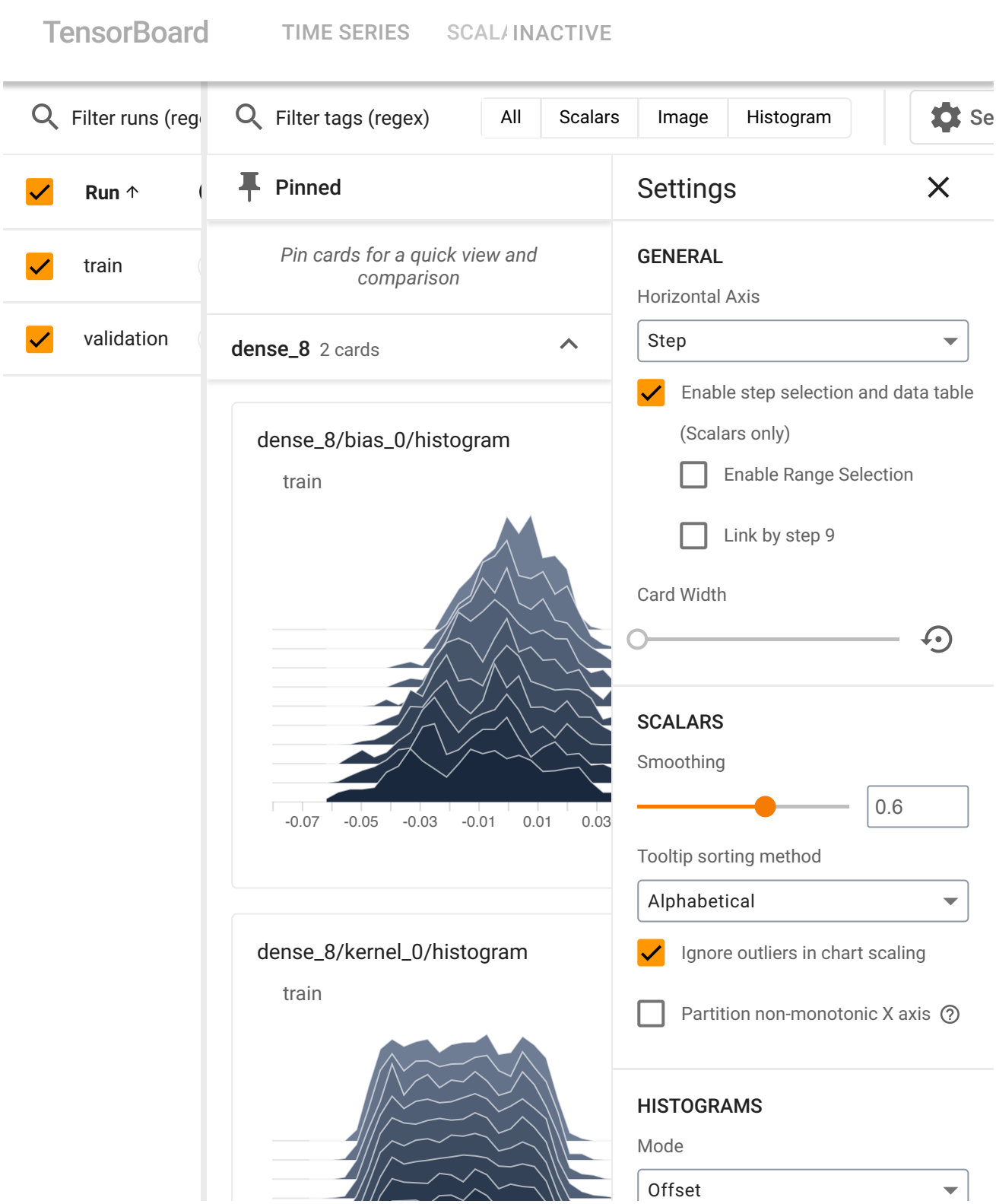
```
model.evaluate(X_train, y_train)
```

```
220/220 [==============================] - 1s 4ms/step - loss: 0.4819 - accura
[0.4818844795227051, 0.7201306819915771]
```

```
model.evaluate(X_val, y_val)
```

```
55/55 [==============================] - 0s 2ms/step - loss: 0.5504 - accuracy
[0.5504295229911804, 0.6312500238418579]
```

```
%tensorboard --logdir={log_folder}
```

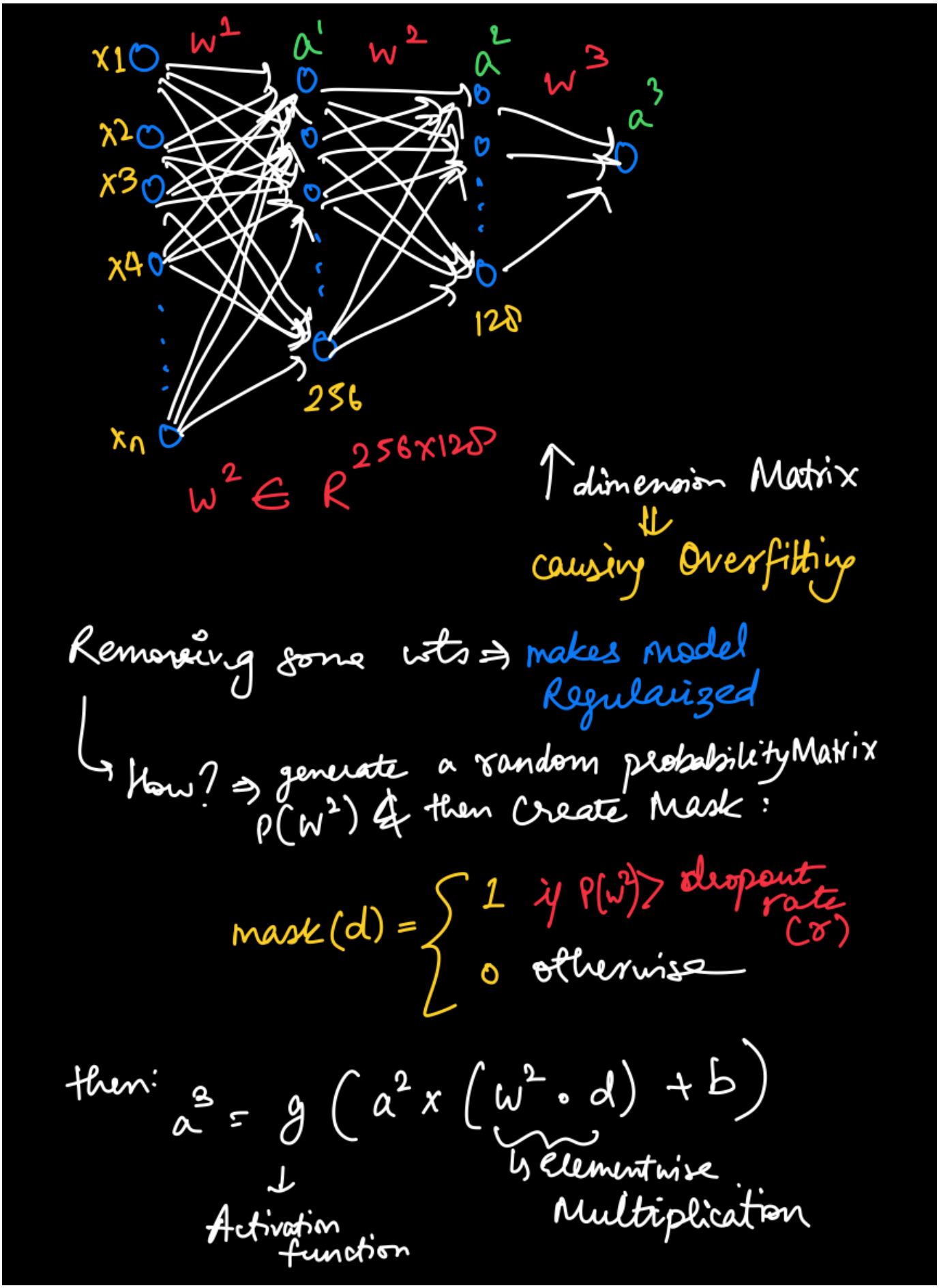


Observe

Even with L2 Regularization:

- the model performance didnot improve drastically

Dropout



What is causing the NN model to overfit ?

Ans: Observe that:

Layer	Descrption
L1	Is the Input Layer
L2	Contains 256 Neurons
L3	Contains 128 Neurons
L4	Contains 64 Neurons
L5	Is the Output Layer

In Layer 2 there are 256 Neurons while Layer 3 contains 128 neurons

What will be the dimension of Weight Matrix W^2 ?

Ans: (256×128)

Now updating weights of such a high dimensional matrix:

- is what making the model complex
- which might be leading the model to overfit

How to resolve this weight update of $W^2 \in R^{256\times128}$ such that NN does not overfit ?

Ans: By not considering / dropping some weights value while computing a^3

How to drop these weights which make up a^3 ?

Ans: By creating a mask matrix (d^2) which contains some randomly generated numbers between (0,1) such that:

- $d^2 = 1$ if, value of $d^2 >$ rate at which we drop the weight of a neuron (r)
- else $d^2 = 0$

Note: This rate r is called as Dropout Rate

What will be the dimension of this mask matrix d^2 ?

Ans: Same as the Weight matrix

- $d^2 \in R^{256\times128}$

How do we calculate a^3 ?

Ans: As $a^3 = g(a^2 \times W^2 + b^2)$, but now we do:

- an **elementwise multiplication** between d^2 and W^2

- $d^2 \circ W^2$

Hence, we can say:

- $a^3 = g(a^2 \times (W^2 \circ d^2) + b^2)$

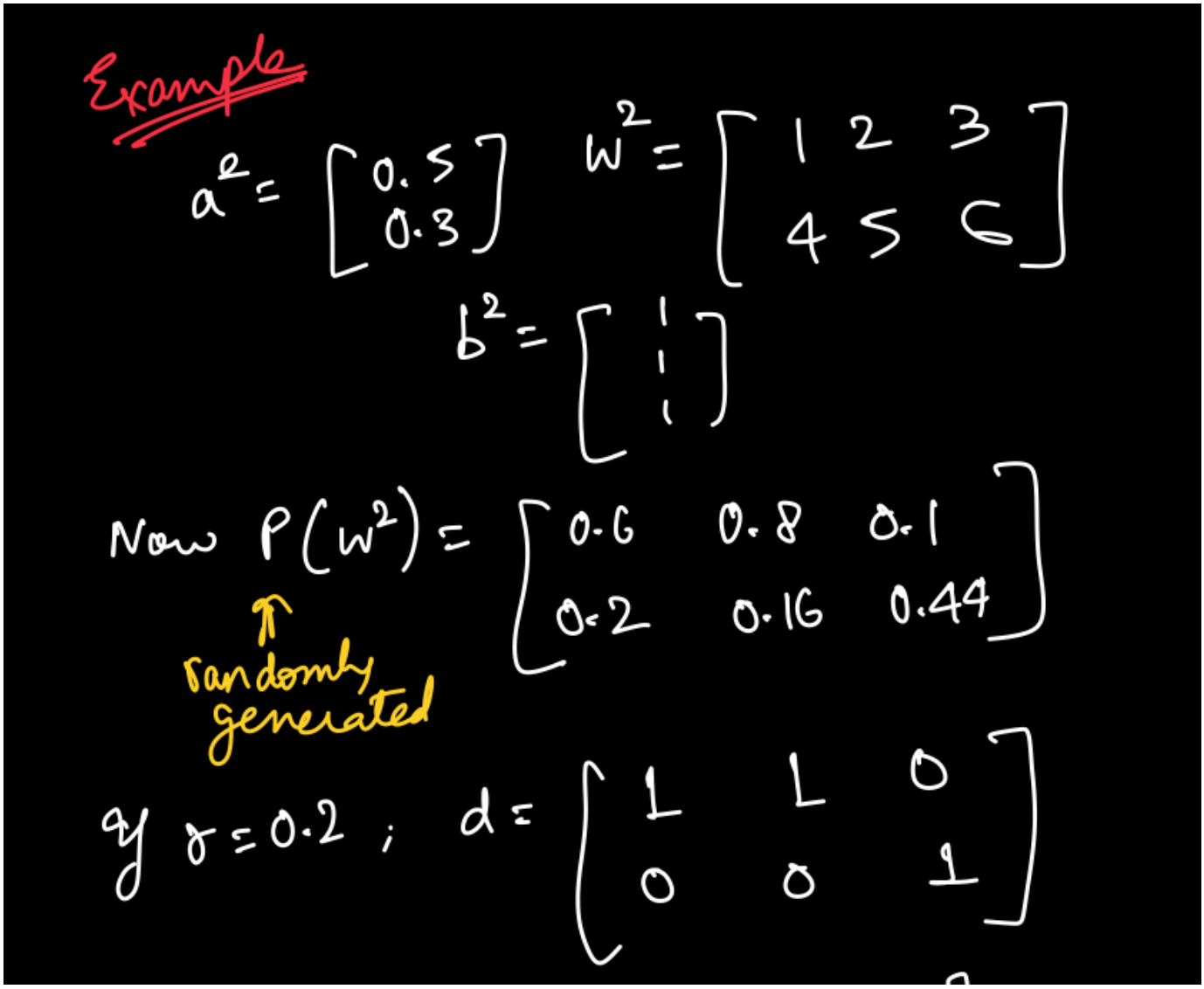
Note: This process of removing/ dropping the weights is called **Edge Dropout**

Instuctor Note

There is another approch to **Dropout**, where the Neurons are dropped instead of the Weights :

- By creating the mask d^2 for the neurons instead of weights,
- And dividing the neurons by a probability factor $p = 1 - r$

Understanding dropout



Lets take an example for better Understanding **Edge Dropout**

Assuming a NN layer has 2 neurons and the following layer has 3 neurons such that:

- $a^2 = [0.5, 0.3]$, $W^2 = [[1, 2, 3], [4, 5, 6]]$ and $b^2 = [1, 1, 1]$

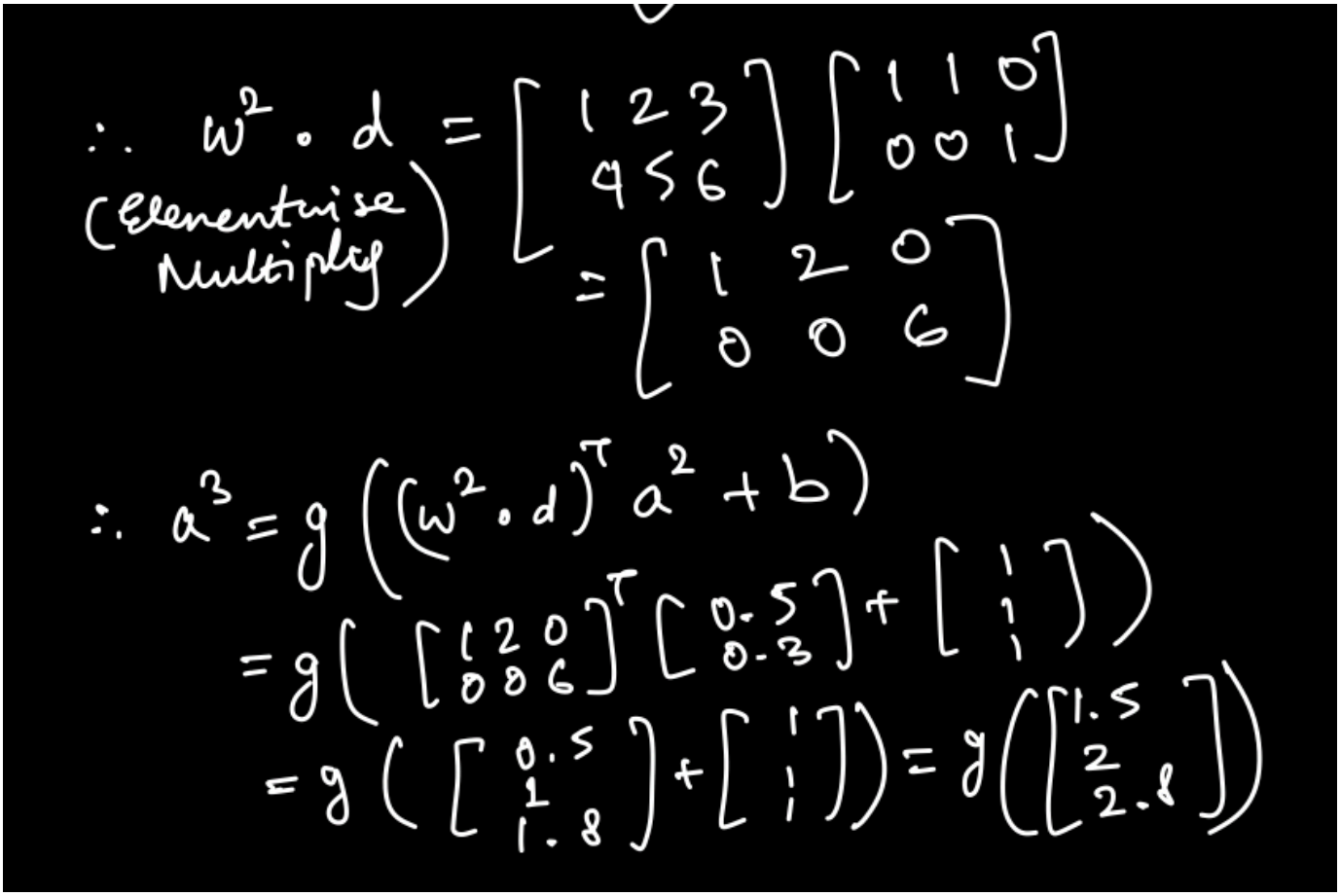
Now if we create a random probability matrix for values of W^2 such that:

- $p(W^2) = [[0.6, 0.8, 0.1], [0.2, 0.16, 0.44]]$

If the Dropout Rate $r = 0.2$, what will be d^2 ?

Ans: Since $d^2 = 1$ if $d^2 > 0.2$, hence:

- $d^2 = [[1, 1, 0], [0, 0, 1]]$



What will be the value of $d^2 \circ W^2$ (Elementwise multiplication) ?

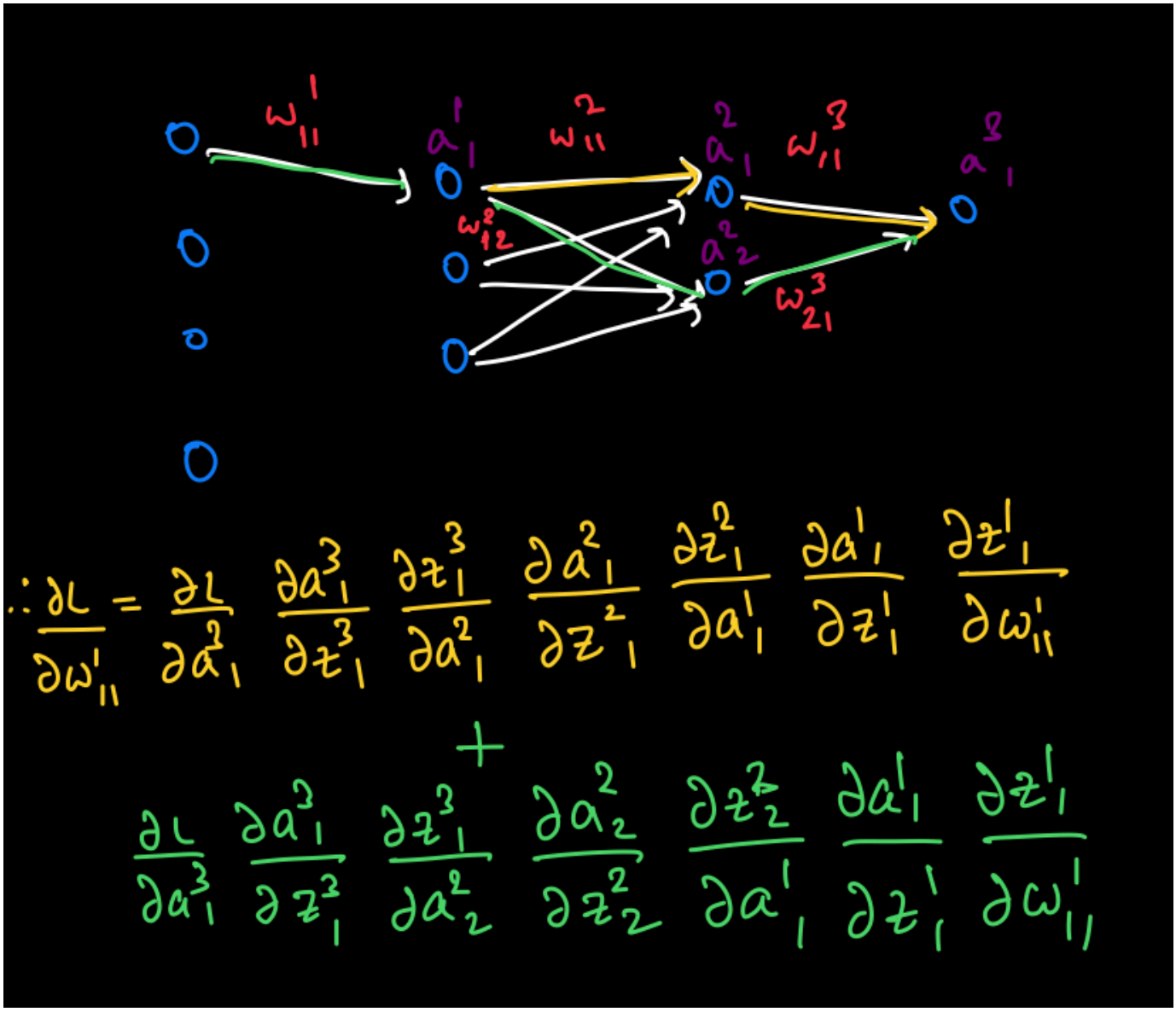
Ans: $d^2 \circ W^2 = [[1, 2, 0], [0, 0, 6]]$

What will be a^3 ?

Ans: $a^3 = g([0.5, 0.3] \times [[1, 2, 0], [0, 0, 6]] + [1, 1, 1])$

On solving, we get:

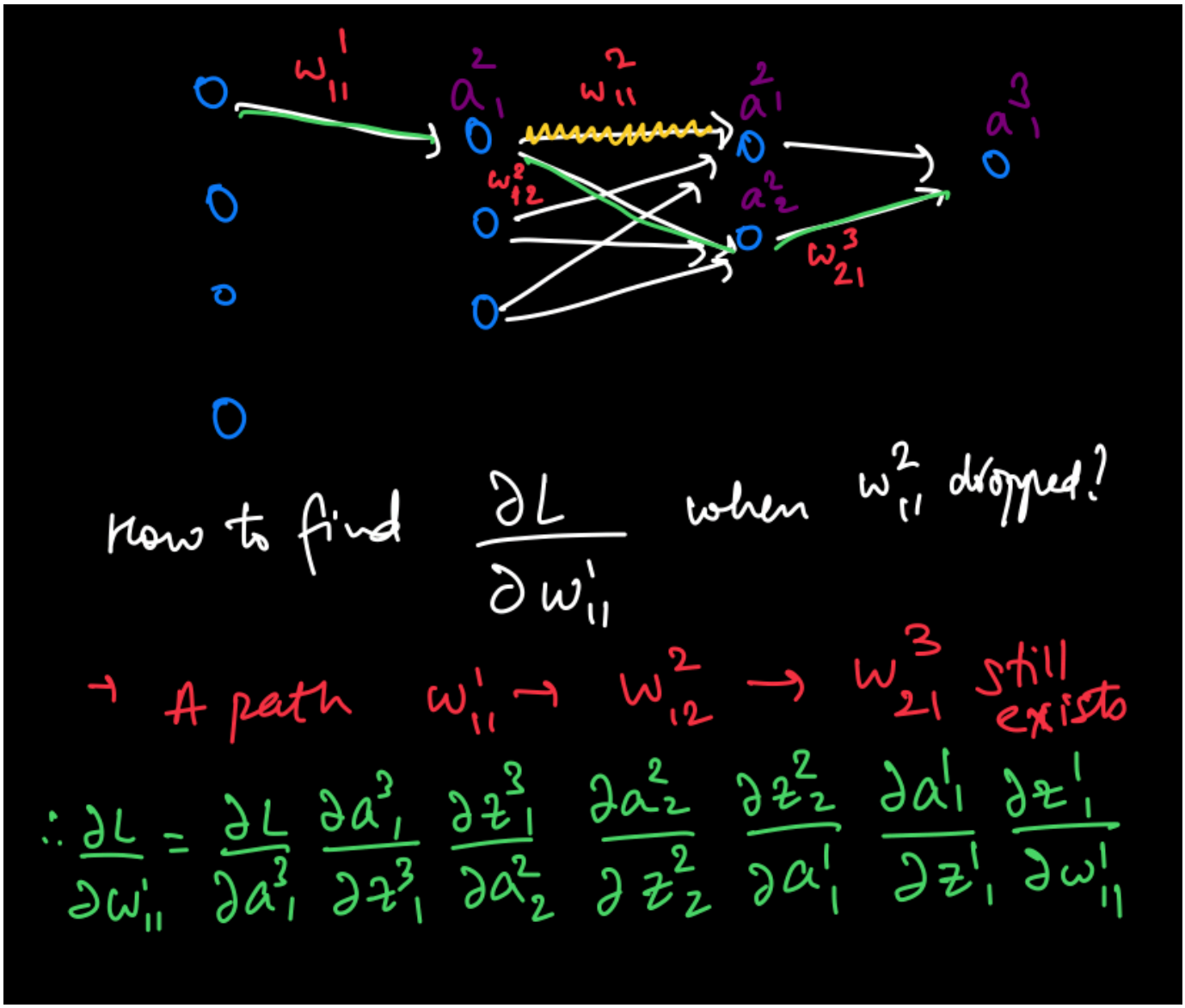
- $a^3 = g([0.5, 1.0, 1.8] + [1, 1, 1]) = g([1.5, 2.0, 2.8])$



- Will the Backpropagation for W_{11}^1 be affected if we drop W_{11}^2 ?

Ans: Lets understand it by doing a backpropagation when W_{11}^2 is not dropped.

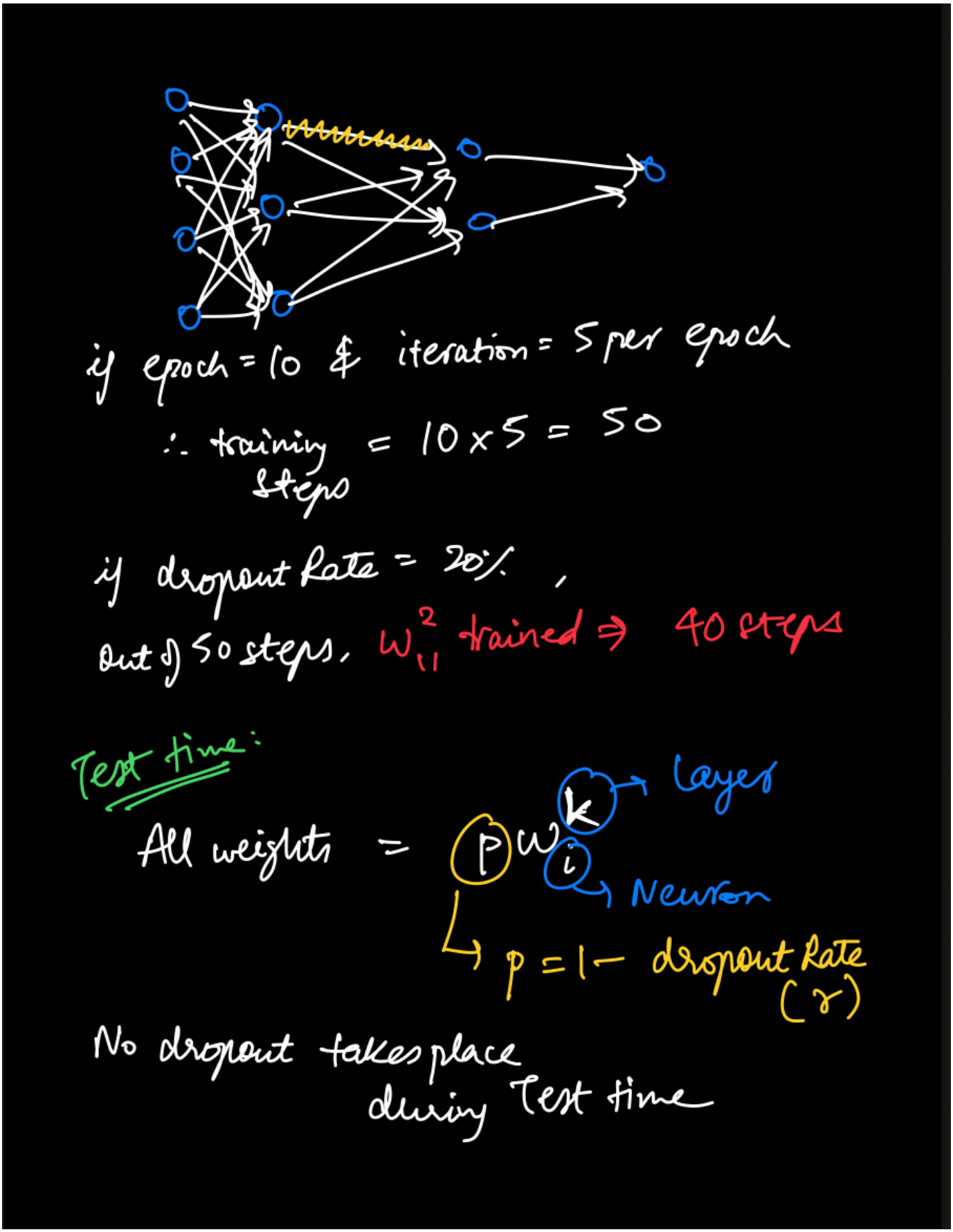
- $\frac{\partial L}{\partial W_{11}^1} = \frac{\partial L}{\partial a_1^3} \times \frac{\partial a_1^3}{\partial z_1^3} \times \frac{\partial z_1^3}{\partial a_1^2} \times \frac{\partial a_1^2}{\partial z_1^2} \times \frac{\partial z_1^2}{\partial a_1^1} \times \frac{\partial a_1^1}{\partial z_1^1} \times \frac{\partial z_1^1}{\partial W_{11}^1} + \frac{\partial L}{\partial a_1^3} \times \frac{\partial a_1^3}{\partial z_1^3} \times \frac{\partial z_1^3}{\partial a_2^2} \times \frac{\partial a_2^2}{\partial z_2^2} \times \frac{\partial z_2^2}{\partial a_1^1} \times \frac{\partial a}{\partial z}$



Now if W_{11}^2 is dropped:

- $\frac{\partial L}{\partial W_{11}^1} = \frac{\partial L}{\partial a_1^3} \times \frac{\partial a_1^3}{\partial z_1^3} \times \frac{\partial z_1^3}{\partial a_2^2} \times \frac{\partial a_2^2}{\partial z_2^2} \times \frac{\partial z_2^2}{\partial a_1^1} \times \frac{\partial a_1^1}{\partial z_1^1} \times \frac{\partial z_1^1}{\partial W_{11}^1}$

Note : Since W_{11}^2 is dropped, the weights are not updated only for this weight value.



If $r = 0.2$, and the model is trained for 10 epochs and it runs 5 iterations per epoch.

Can we say, that the weights which are dropped gets their weight updated for all $10 \times 5 = 50$ training steps ?

Ans: No, because of the fact there is a 20% drop rate:

- The weights are updated for only $50 \times (1 - 0.2) = 40$ training steps
- This dropping of weights helps the neurons to not be inclinced towards any one particular feature
    - as that feature weight can be dropped off
- Thus regularizing the model by giving importance to every neuron/feature.

Then how during test time, the weights of the model accomidate this 40 training step ?

Ans: During the test time,

- Each weight of the model are multiplied with $p = 1 - r$

Will there be dropout during test time ?

Ans: No

Quiz

```
if there are 2 hidden layer neurons such that W = [0.4, 0.2, 0.7, 0.01] and the mask vect
```

```
1. Neuron 1 and 3
2. Neuron 2 and 3
3. Neuron 2 and 4
```

Answer

3. Neuron 2 and 4

Quiz

```
if due to dropout, weight value is updated for only 20 steps out of 30 , then what is the
```

```
1. 66.66%
2. 33.33%
3. 55.33%
```

Answer

2. 33.33%

Explanation

$$30 - 30 \times rate = 20$$

- $30 \times rate = 10$
- $rate = \frac{10}{30} = 0.3333$

With this lets now implement Dropout using Tensorflow Keras

```
from tensorflow.keras.layers import Dropout
def create_Dropout():
    # lambda = 0.01
    L2Reg = tf.keras.regularizers.L2(l2=1e-6)
    model = Sequential([
                    Dense(256, activation="relu", kernel_regularizer = L2Reg ),
                    Dropout(0.3),
                    Dense(128, activation="relu", kernel_regularizer = L2Reg),
                    Dropout(0.3),
                    Dense(64, activation="relu", kernel_regularizer = L2Reg),
                    Dense(1 , activation = 'sigmoid')])
    return model


model = create_Dropout()

model.compile(optimizer = tf.keras.optimizers.Adam(),
                loss = tf.keras.losses.BinaryCrossentropy(),
                metrics=["accuracy"])



!rm -rf logs


from tensorflow.keras.callbacks import TensorBoard

now = datetime.now()
log_folder = "tf_logs/.../" + now.strftime("%Y%m%d-%H%M%S") + "/"

tb_callback = TensorBoard(log_dir=log_folder, histogram_freq=1)
```

Training the model

```
from tensorflow.keras.callbacks import TensorBoard

now = datetime.now()
log_folder = "tf_logs/.../" + now.strftime("%Y%m%d-%H%M%S") + "/"

tb_callback = TensorBoard(log_dir=log_folder, histogram_freq=1)
```

Evaluating the model

```
model.evaluate(X_train, y_train)
```

```
220/220 [==============================] - 1s 2ms/step - loss: 0.6859 - accura
[0.6858837008476257, 0.5547662973403931]
```

```
model.evaluate(X_val, y_val)
```

```
55/55 [==============================] - 0s 2ms/step - loss: 0.6829 - accuracy
[0.6829455494880676, 0.5704545378684998]
```

```
%tensorboard --logdir={log_folder}
```

TensorBoard INACTIVE

No dashboards are active for the current data set.

Probable causes:

- You haven't written any data to your event files.
- TensorBoard can't find your event files.

If you're new to using TensorBoard, and want to find out how to add data and set up your event files, check out the README and perhaps the TensorBoard tutorial.

If you think TensorBoard is configured properly, please see the section of the README devoted to missing data problems and consider filing an issue on GitHub.

Last reload: Apr 24, 2024, 5:40:31 PM

Log directory: tf_logs/.../20240103-160519/

Observe

using Dropout and Regularization,

- the model did reduce its overfitting

Batch Normalization

What other Hyperparameter tuning techniques we can apply to make the model perform better ?

Ans: Recall that, we always standardize the inputs to the model for Gradient Descent to quickly reach global minima.

Now if we have a four layer NN, such that:

- the output of Layer2 (a^2) becomes input for layer 3
- With weight matrix as W^3 with bias as b^3

Will the input for layer 3 still be in normalized form ?

Ans: **NO**. Clearly, the input to each layer is affected by the weights and biases of the previous layers.

So now imagine, there due to data being not normalizied

- there is a 0.1% change in data distribution

What will happen if this change in distribution goes down a 100 layer Neural Network ?

Ans: This 0.1% change gets amplified when going down the network

- This leads to change in the input distribution to hidden layers of the network
- and the weights updation gets impacted

This is whats called an **internal covariate shift**

Can we normalize a^2 so to make w^3 and b^3 optimum ?

Ans: Yes we can, now if we consider m number of neurons in Layer 2,

- then $z_i^2 . . . . . z_m^2$ becomes the neuron output before the activation function

Therefore we can say:

- the mean $\mu = \frac{1}{m} \sum_{i=1}^{i=m} z_i$
- the variance $\sigma^2 = \frac{1}{m} \sum_{i=1}^{i=m} (z_i - \mu)^2$

Then the normalized z_i ($znorm_i$) becomes:

- $znorm_i = \frac{z_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$ Where ϵ is a very small value $1e^{-10}$ to prevent the denominator to turn 0 when variance is 0

but do we want to have all the hidden layers to have outputs with exact mean = 0 and variance = 1 ?

Ans: No, because if two hidden layers have the same distribution of mean = 0 and variance = 1

- the information learnt about the data for both the layers will be almost same

Thus making the use of the 2nd hidden layer redundant.

Instructor Note: Please read Expressive-Power-Of-NeuralNetwork for proof

how to make the distribution slightly different for each hidden layer ?

Ans: By scaling and shifting the $znorm_i$ by using two learnable parameters (γ, β) such that:

- $\hat{z_i} = \gamma \times znorm_i + \beta$

Note: This Entire process of normalization with scaling and shifting is known as **Batch Normalization**

Quiz

```
Batch Normalization helps in faster convergence of model

1. True
2. False
```

Answer

1. True

Lets now implement Batch Normalization in the Baseline Model

```
from tensorflow.keras.layers import BatchNormalization
from tensorflow.keras.layers import Activation
from tensorflow.keras.activations import relu

def create_BatchNormalization_model():
    L2Reg = tf.keras.regularizers.L2(l2=1e-6)
    model = Sequential([
                    Dense(256, kernel_regularizer = L2Reg),
                    BatchNormalization(),
                    Activation(relu),
                    Dropout(0.2),
                    Dense(128, kernel_regularizer = L2Reg),
                    BatchNormalization(),
                    Activation(relu),
                    Dense(64,kernel_regularizer = L2Reg ),
                    BatchNormalization(),
                    Activation(relu),
                    Dense(1)])
    return model


model = create_BatchNormalization_model()

model.compile(optimizer = tf.keras.optimizers.Adam(learning_rate=0.001),
                loss = tf.keras.losses.BinaryCrossentropy(),
                metrics=["accuracy"])


!rm -rf logs


from tensorflow.keras.callbacks import TensorBoard

now = datetime.now()
log_folder = "tf_logs/.../" + now.strftime("%Y%m%d-%H%M%S") + "/"

tb_callback = TensorBoard(log_dir=log_folder, histogram_freq=1)
```

Increasing epoch to 15 now

```
history = model.fit(X_train, y_train, validation_data = (X_val, y_val),  epochs=1
```

Evaluting the model

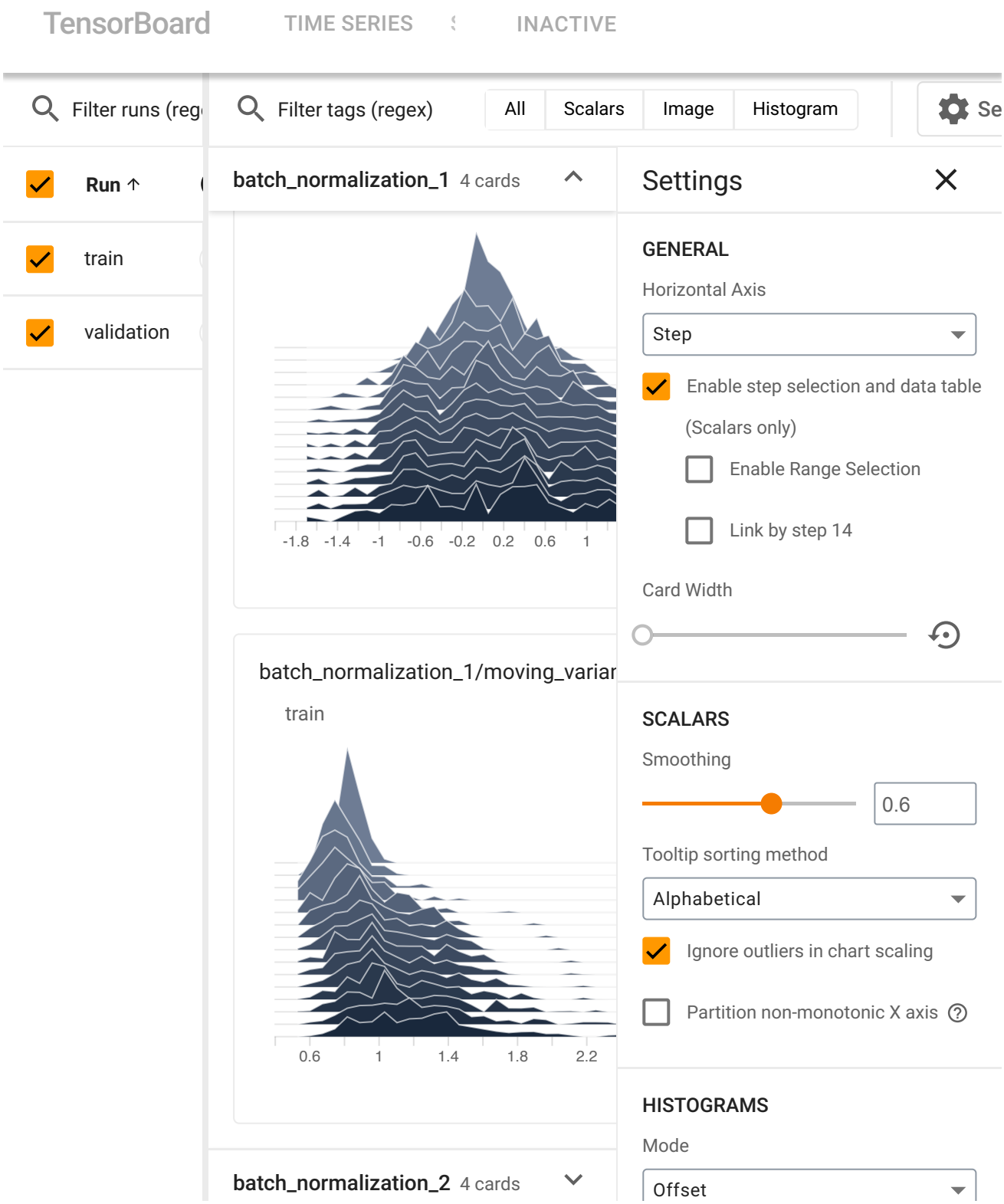
```
model.evaluate(X_train, y_train)
```

```
220/220 [==============================] - 1s 2ms/step - loss: 1.6455 - accura
[1.6455154418945312, 0.6378747224807739]
```

```
model.evaluate(X_val, y_val)
```

```
55/55 [==============================] - 0s 2ms/step - loss: 1.6294 - accuracy
[1.629418134689331, 0.6181818246841431]
```

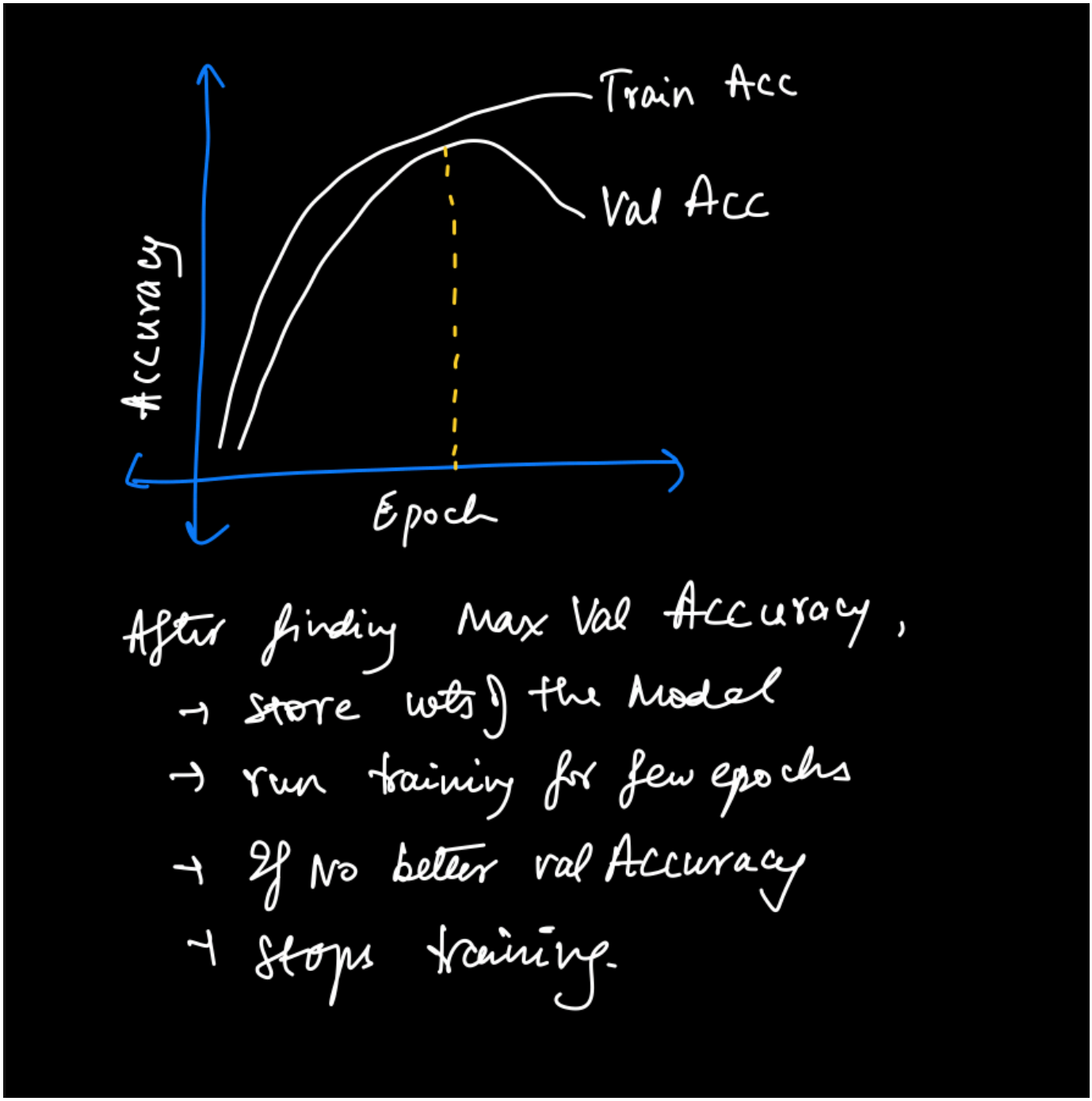
```
%tensorboard --logdir={log_folder}
```



We see after applying Batch Normalization :

- L2 Regularization worked
- And made the model not overfit the data

Early Stopping



Observe

When using Batch Normalization,

- around epoch 8 the Validation Accuracy is maximum,
- and then it starts to decrease

Can there be a way to make the model not update the weights after the validation accuracy reaches maximum ?

Ans: Yes, by using callbacks to stop the training when:

- The validation accuracy starts decreasing
- or validation loss starts to increase

How to know if the model has reached maximum validation accuracy or not?

Ans: By using callbacks such that:

- One callback stores the weights of model when best validation accuracy is attained
- While the other callback, Stops the training:
    - after not finding a better validation accuracy, when the model is trained for some epoch (τ)

Note: Keras uses `EarlyStoppingCallback` for:

- stopping the training after no better Validation accuracy attained when ran for some epoch (τ)

Also keras uses another callback `ModelCheckpointCallback` for:

- Storing the weights of the model which had the best Validation Accuracy

```
model = create_BatchNormalization_model()

model.compile(optimizer = tf.keras.optimizers.Adam(learning_rate=0.001),
              loss = tf.keras.losses.BinaryCrossentropy(),
              metrics=["accuracy"])


!rm -rf logs


from tensorflow.keras.callbacks import TensorBoard

now = datetime.now()
log_folder = "tf_logs/.../" + now.strftime("%Y%m%d-%H%M%S") + "/"

tb_callback = TensorBoard(log_dir=log_folder, histogram_freq=1)


EarlyStoppingCallback = tf.keras.callbacks.EarlyStopping(monitor='val_accuracy',
ModelCheckpointCallback = tf.keras.callbacks.ModelCheckpoint(filepath='tf_model.h
                                                             monitor='val_accurac
                                                             save_best_only=True,
                                                             mode='max')


history = model.fit(X_train, y_train, validation_data = (X_val, y_val),  epochs=1
```

```
/usr/local/lib/python3.10/dist-packages/keras/src/engine/training.py:3103: Us
  saving_api.save_model(
```

```
%tensorboard --logdir={log_folder}
```